

图像向量检索实践作业

2026 年 5 月 19 日

1 作业简介与评估指标

本次实践作业要求同学完成一个小型图像向量检索流程。基本思想是：先选择一个图像数据集，然后使用图像或多模态 embedding 模型将图片转换为高维向量，再使用 FAISS 等开源 ANNS 库构建近似最近邻索引，最后对 query 图片进行检索并分析结果。

本作业关注的核心问题包括：

1. 如何将图片转换为向量表示；
2. 如何将全量向量划分为 base 向量集合与 query 向量集合；
3. 如何生成 ground truth；
4. 如何使用 FAISS 中的不同 ANNS 索引完成向量搜索；
5. 如何统计索引构建时间、索引大小、QPS 与 Recall@10；
6. 如何对检索结果进行可视化与分析。

评估指标 Recall@10 定义

本作业使用 Recall@10 评估某种 ANNS 方法返回结果与 ground truth 的一致程度。

对于某一条 query，设：

- R_{10} 表示某种 ANNS 方法返回的前 10 个结果集合；
- G_{10} 表示 ground truth 中的前 10 个结果集合。

则该 query 的 Recall@10 定义为：

$$\text{Recall@10} = \frac{|R_{10} \cap G_{10}|}{10}$$

例如，某种 ANNS 方法返回的前 10 张图片中，有 8 张也出现在 ground truth 的前 10 张图片中，则该 query 的 Recall@10 为：

$$\text{Recall@10} = \frac{8}{10} = 0.8$$

多条 query 的平均 Recall@10 为所有 query 的 Recall@10 的平均值：

$$\text{MeanRecall@10} = \frac{1}{N} \sum_{i=1}^N \text{Recall@10}_i$$

其中 N 为 query 数量。

评估指标 QPS 定义

QPS，全称为 Queries Per Second，表示系统每秒能够完成多少条 query 的检索。QPS 用于评估索引方法的查询效率。

设：

- N_q 表示 query 数量；
- T 表示完成全部 query 检索所用的总时间，单位为秒。

则 QPS 定义为：

$$\text{QPS} = \frac{N_q}{T}$$

例如，若共有 10 条 query，完成全部检索用时 0.02 秒，则：

$$\text{QPS} = \frac{10}{0.02} = 500$$

QPS 越高，说明查询速度越快。但需要注意，QPS 需要结合 Recall@10 一起分析：某些索引方法可能查询速度很快，但 Recall@10 较低；某些方法 Recall@10 较高，但查询速度较慢。

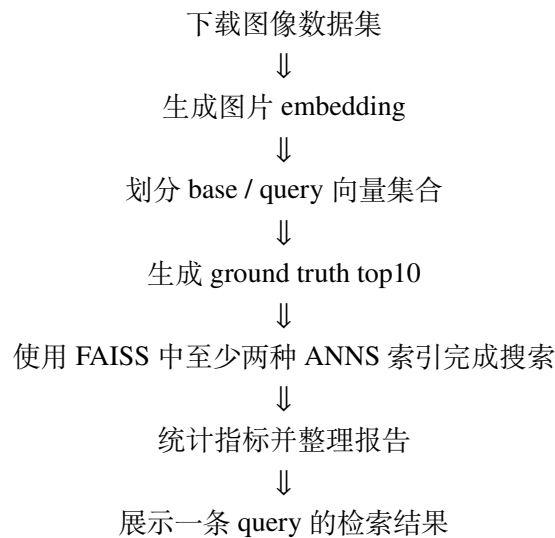
2 最小完成要求

本作业只要求提交实验报告，不要求提交代码文件。报告中需要完整说明实验流程、实验设置、实验结果与分析。

最小完成要求如下：

1. 下载或整理 1 个图像数据集；
2. 使用 1 种 embedding 方法将图片转换为向量；
3. 将全量向量划分为 base 向量集合与 query 向量集合；
4. 使用精确检索方法计算 ground truth top10；
5. 使用 FAISS 中至少 2 种 ANNS 索引方法完成搜索；
6. 统计索引构建时间、索引大小、QPS、Recall@10；
7. 任选 1 条 query，将检索结果整理成可视化示例图；
8. 在报告中进行结果分析。

最小工作流程如下：



2.1 步骤 1：下载图像数据集

同学可以任选图片类数据集完成实验。以下提供一些参考选择，也可以选择其他适合单机处理的图像数据集。

数据集名称	简介	图片数量及占用空间	是否标注	下载链接
Oxford-IIIT Pet	宠物图片数据集, 包含猫狗品种图片, 图片存在姿态、尺度和光照变化, 适合做相似宠物图片检索。	37 个类别, 每类约 200 张; 官方图片包约数百 MB。	有 breed、head ROI、trimap segmentation 等标注。	Oxford Pets 官方页面
CIFAR-10	经典小型图像分类数据集, 图片较小, 适合快速测试向量生成和检索流程。	60,000 张 32×32 彩色图片; Python 版本约 163 MB。	有 10 类类别标注。	CIFAR 官方页面
Food-101	食物图片数据集, 适合做食物相似图片检索。图片内容直观, 适合展示。	101 类, 共 101,000 张图片; 官方下载约 5 GB。	有食物类别标注。	Food-101 官方页面
Caltech-101	经典物体类别图像数据集, 类别多, 图片规模适中。	约 9,000 张图片; 101 类, 每类约 40–800 张。	有类别标注与物体轮廓标注。	Caltech-101 页面

报告中需要说明：

- 最终选择的数据集名称；
- 数据集来源；
- 使用的图片数量；
- 数据集是否包含标注；

- 若只使用了数据集子集，需要说明抽取方式。
若数据集较大，可以只抽取其中一部分图片完成实验，建议至少使用 5000 张图片。

2.2 步骤 2：将数据 embedding 为向量，划分 base 与 query 向量集合，并计算 ground truth

2.2.1 Embedding 方式选择

本作业中，embedding 是指将图片转换为向量表示的过程。同学可以选择本地 embedding 模型，也可以选择调用云端 embedding API。

方式一：本地 embedding 模型 本地 embedding 模型适合在个人电脑或服务器上直接运行。以下提供一些参考选择。

模型	类型	是否支持文本	简介
ResNet50	图像模型	否	经典卷积神经网络，可使用 ImageNet 预训练权重，去掉最后分类层后得到图像特征向量。
CLIP ViT-B/32	图 文 多 模 态 模型	是	CLIP 使用图像编码器和文本编码器，将图片和文本映射到同一向量空间，适合图搜图和文本搜图。
DINOv2-small/base	自 监 督 视 觉 模型	否	DINOv2 是视觉基础模型，适合提取图像特征并用于图像相似检索。
SigLIP / SigLIP2	图 文 多 模 态 模型	是	SigLIP 是 CLIP 类图文模型，可以将图像和文本映射到可比较的向量空间。
ConvNeXt / ViT	图像模型	否	现代视觉模型，可用于提取图片特征，适合与其他图像 embedding 方法进行比较。

本地 embedding 关键代码示例 下面以 CLIP ViT-B/32 为例，展示如何将图片转换为向量。实际实验中，同学需要将该过程扩展到整个图片文件夹，并将生成的向量统一保存为 fvecs 文件。write_fvecs 函数见附录。

运行前需要安装依赖：

```
pip install torch transformers pillow numpy
```

关键代码如下：

```
import numpy as np
import torch
from PIL import Image
from transformers import CLIPImageProcessor, CLIPVisionModel

# 步骤 1: 选择运行设备
# 如果本机有 GPU，则使用 cuda；否则使用 CPU。
```



```
device = "cuda" if torch.cuda.is_available() else "cpu"

# 步骤 2: 加载 CLIP 图像编码器
# processor 负责图片预处理; model 负责把图片编码为向量。
model_name = "openai/clip-vit-base-patch32"
processor = CLIPImageProcessor.from_pretrained(model_name)
model = CLIPVisionModel.from_pretrained(model_name).to(device)
model.eval() # 推理模式, 不进行训练

# 步骤 3: 读取一张图片
# 实际实验中, 需要循环读取整个 images 文件夹中的所有图片。
image_path = "data/images/example.jpg"
img = Image.open(image_path).convert("RGB")

# 步骤 4: 图片预处理
# processor 会完成 resize、crop、normalize 等模型所需的预处理。
inputs = processor(images=img, return_tensors="pt")
pixel_values = inputs["pixel_values"].to(device)

# 步骤 5: 前向传播, 得到图片 embedding
# pooler_output 可作为整张图片的向量表示。
with torch.no_grad():
    outputs = model(pixel_values=pixel_values)
    emb = outputs.pooler_output # shape: [1, 768]

# 步骤 6: L2 归一化
# 归一化后, 后续使用 L2 距离或余弦相似度时更加稳定。
emb = torch.nn.functional.normalize(emb, p=2, dim=1)

# 步骤 7: 转换为 numpy 的 float32 向量
# 后续 FAISS 建索引通常要求输入为 float32。
vector = emb.cpu().numpy().astype("float32")

print("embedding shape:", vector.shape)

# 步骤 8: 批量处理时, 将所有图片向量收集为矩阵后写入 fvecs
# all_vectors 的形状应为 [N, d], N 为图片数量, d 为向量维度。
# write_fvecs("pet_all.fvecs", all_vectors)
```

方式二: 调用 embedding API 同学也可以使用云端 embedding API 完成向量化。以下提供一个参考选择。

服务	是否支持图片 embedding	是否支持图文统一向量空间	简介
阿里云百炼 / DashScope 多模态向量	支持	支持	阿里云多模态 embedding 模型可将文本、图像、视频等输入转换为向量表示, 可用于以图搜图、文本搜图、图文检索等任务。参考: 阿里云 Model Studio Embedding 文档

报告中需要说明:

- 使用的 embedding 模型或 API;
- 输入数据类型;
- 输出向量维度;
- 向量数据类型, 例如 float32;
- 是否进行了 L2 normalization;
- 是否支持文本搜图。

2.2.2 划分 base 与 query, 并计算 ground truth

全量图片 embedding 完成后, 需要划分为两个集合:

- base 向量集合: 用于建立索引和被检索;
- query 向量集合: 用于发起检索查询。

要求如下:

1. 随机选取一小部分图片向量作为 query;
2. 其余图片向量作为 base;
3. query 向量不得出现在 base 中;
4. ground truth 使用精确检索方法计算;
5. 本作业要求计算 ground truth top10。

推荐输出文件如下:

文件名	说明
{dataset}_base.fvecs	base 向量集合
{dataset}_query.fvecs	query 向量集合
{dataset}_base_path.csv	base 向量编号与对应图片路径
{dataset}_query_path.csv	query 向量编号与对应图片路径
{dataset}_groundtruth.ivecs	query 对应的 ground truth top10 base 编号

报告中不需要提交上述文件, 但需要说明向量存储格式和 ground truth 的生成方式。本作业统一建议: 向量文件使用 fvecs 格式, ground truth 文件使用 ivecs 格式。

base/query 划分与保存关键代码示例 下面代码假设已经得到全量图片向量 vectors 和对应图片路径 image_paths。其中, write_fvecs 函数见附录。



```
import csv
import random
import numpy as np

# vectors: shape [N, d], dtype float32
# image_paths: length N, each item is the corresponding image path

def write_path_csv(path, paths):
    """保存向量编号与图片路径的对应关系。"""
    with open(path, "w", encoding="utf-8", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["vector_id", "path"])

        # vector_id 是该图片在对应 fvecs 文件中的行号。
        for i, p in enumerate(paths):
            writer.writerow([i, p])

def split_and_save(vectors, image_paths, query_size=10, seed=42):
    """将全量向量划分为 query 和 base, 并保存为 fvecs。"""
    # 步骤 1: 确保向量数据类型为 float32, 便于后续 FAISS 使用。
    vectors = np.asarray(vectors, dtype="float32")
    n = vectors.shape[0]

    # 步骤 2: 固定随机种子, 保证每次实验划分结果可复现。
    random.seed(seed)

    # 步骤 3: 随机选择 query_size 条作为 query。
    query_ids = sorted(random.sample(range(n), query_size))
    query_id_set = set(query_ids)

    # 步骤 4: 其余向量作为 base, 注意 query 不得出现在 base 中。
    base_ids = [i for i in range(n) if i not in query_id_set]

    # 步骤 5: 按照编号取出 query/base 向量与图片路径。
    query_vectors = vectors[query_ids]
    base_vectors = vectors[base_ids]
    query_paths = [image_paths[i] for i in query_ids]
    base_paths = [image_paths[i] for i in base_ids]

    # 步骤 6: 保存向量文件和路径映射文件。
    write_fvecs("pet_query.fvecs", query_vectors)
    write_fvecs("pet_base.fvecs", base_vectors)
    write_path_csv("pet_query_path.csv", query_paths)
    write_path_csv("pet_base_path.csv", base_paths)
```



```
return query_vectors, base_vectors
```

ground truth 生成关键代码示例 下面代码使用精确检索 IndexFlatL2 生成 ground truth top10。read_fvecs 与 write_ivecs 函数见附录。

```
import faiss
import numpy as np

def generate_groundtruth(
    query_fvecs_path="pet_query.fvecs",
    base_fvecs_path="pet_base.fvecs",
    output_ivecs_path="pet_groundtruth.ivvecs",
    k=10,
):
    """使用精确检索生成 ground truth top-k."""
    # 步骤 1: 读取 query 向量和 base 向量。
    query_vectors = read_fvecs(query_fvecs_path).astype("float32")
    base_vectors = read_fvecs(base_fvecs_path).astype("float32")

    # 步骤 2: 检查 query 和 base 的向量维度是否一致。
    nq, dq = query_vectors.shape
    nb, db = base_vectors.shape

    if dq != db:
        raise ValueError("query and base vectors have different dimensions")
    if k > nb:
        raise ValueError("k cannot be larger than the number of base vectors")

    # 步骤 3: 使用 IndexFlatL2 建立精确检索索引。
    # IndexFlatL2 会对 query 与所有 base 向量计算 L2 距离。
    index = faiss.IndexFlatL2(db)
    index.add(base_vectors)

    # 步骤 4: 对每条 query 搜索 top-k 个最近邻。
    distances, indices = index.search(query_vectors, k)

    # 步骤 5: 将最近邻编号保存为 ivecs 文件。
    # indices 中的编号是 base 向量集中的编号。
    write_ivecs(output_ivecs_path, indices.astype("int32"))

    print("ground truth saved to:", output_ivecs_path)
    print("query count:", nq, "base count:", nb, "topK:", k)
```

注意: pet_groundtruth.ivvecs 中存储的是 base 向量编号, 而不是原始全量图片编号。

2.3 步骤 3: 使用 FAISS ANNS 库完成搜索, 统计索引大小、索引构建时间、QPS 与 Recall

本步骤要求使用 FAISS 中的 ANNS 索引完成搜索。至少选择两种不同索引方法完成实验, 不包括生成 ground truth 时使用的 IndexFlatL2。

可选 FAISS 索引方法

索引方法	FAISS 类	是否需 要训练	说明
Flat 精确检索	IndexFlatL2	否	精确暴力检索方法。会计算 query 与 base 中所有向量的距离或相似度。本作业中主要用于生成 ground truth, 不作为 ANNS 方法参与对比。
IVF	IndexIVFFlat	是	倒排文件索引。先使用聚类方法将 base 向量划分到多个倒排桶中, 查询时只访问部分桶, 从而减少距离计算量。主要参数包括 nlist 和 nprobe。
HNSW	IndexHNSWFlat	否	图索引方法。通过构建层次化近邻图, 在图上进行近似搜索。主要参数包括 M、efConstruction、efSearch。
LSH	IndexLSH	否	局部敏感哈希方法。通过哈希编码近似表示向量相似性, 再基于哈希结果进行近似检索。主要参数为 nbits。
PQ	IndexPQ	是	乘积量化方法。将原始向量切分为多个子向量, 并分别进行量化编码, 从而降低向量存储空间。适合分析压缩率、索引大小与 Recall 之间的关系。
IVF-PQ	IndexIVFPQ	是	IVF 与 PQ 的组合方法。先使用 IVF 将向量划分到不同倒排桶中, 再使用 PQ 对桶内向量进行压缩编码。相比 IndexIVFFlat, IndexIVFPQ 通常可以显著降低索引大小, 但可能带来一定 Recall 损失。主要参数包括 nlist、nprobe、m、nbits。
IVF-RaBitQ	版本相关	是	IVF 与低比特量化思想结合的方法, 适合进阶调研量化型 ANNS 方法。不同 FAISS 版本对该方法的支持情况可能不同, 若本地环境没有对应接口, 可将其作为调研项。

报告中需要说明：

- 选择了哪些 FAISS 索引方法；
- 每种方法的主要参数；
- 每种方法的索引构建时间；
- 每种方法的索引占用空间；
- 每种方法的 QPS；
- 每种方法的 Recall@10。

FAISS 检索关键代码示例 下面以 IndexIVFFlat 为例，展示如何统计索引构建时间、索引大小、QPS 和 Recall@10。read_fvecs 与 read_ivecs 函数见附录。

```
import os
import time
import faiss
import numpy as np

def recall_at_10(result_ids, gt_ids):
    """计算单条 query 的 Recall@10。"""
    # result_ids 是 ANNS 方法返回的 top10 编号。
    result_set = set(int(x) for x in result_ids[:10])

    # gt_ids 是精确检索得到的 ground truth top10 编号。
    gt_set = set(int(x) for x in gt_ids[:10])

    # Recall@10 = 命中的 ground truth 数量 / 10。
    return len(result_set & gt_set) / 10.0

def mean_recall_at_10(all_results, all_gt):
    """计算所有 query 的平均 Recall@10。"""
    values = []
    for i in range(len(all_results)):
        values.append(recall_at_10(all_results[i], all_gt[i]))
    return sum(values) / len(values)

def run_ivf_search(
    base_fvecs_path="pet_base.fvecs",
    query_fvecs_path="pet_query.fvecs",
    gt_ivecs_path="pet_groundtruth.ivecs",
    index_save_path="ivf.index",
    topk=10,
):
    # 步骤 1: 读取 base、query 和 ground truth 文件。
    base_vectors = read_fvecs(base_fvecs_path).astype("float32")
```



```
query_vectors = read_fvecs(query_fvecs_path).astype("float32")
gt_indices = read_ivecs(gt_ivecs_path)

# 步骤 2: 检查向量数量和维度。
nb, d = base_vectors.shape
nq, dq = query_vectors.shape
if d != dq:
    raise ValueError("base and query vectors have different dimensions")

# 步骤 3: 设置 IVF 参数。
# nlist 表示聚类中心 / 倒排列表数量。
# nprobe 表示查询时访问多少个倒排列表。
nlist = 100
nprobe = 10

# 步骤 4: 创建 IVFFlat 索引。
# quantizer 用于把向量分配到最近的聚类中心。
quantizer = faiss.IndexFlatL2(d)
index = faiss.IndexIVFFlat(quantizer, d, nlist, faiss.METRIC_L2)

# 步骤 5: 训练索引并加入 base 向量, 同时统计构建时间。
# IVF 需要先 train, 再 add。
t0 = time.perf_counter()
index.train(base_vectors)
index.add(base_vectors)
build_time_ms = (time.perf_counter() - t0) * 1000

# 步骤 6: 设置查询时的 nprobe。
# nprobe 越大, Recall 通常越高, 但查询速度可能变慢。
index.nprobe = nprobe

# 步骤 7: 保存索引, 并统计索引文件大小。
faiss.write_index(index, index_save_path)
index_size_mb = os.path.getsize(index_save_path) / 1024 / 1024

# 步骤 8: 执行检索, 并统计查询时间与 QPS。
t0 = time.perf_counter()
distances, result_indices = index.search(query_vectors, topk)
search_time_sec = time.perf_counter() - t0
qps = nq / search_time_sec

# 步骤 9: 与 ground truth 比较, 计算 Recall@10。
recall10 = mean_recall_at_10(result_indices, gt_indices)

# 步骤 10: 统计 base 向量矩阵本身的内存大小。
base_size_mb = base_vectors.nbytes / 1024 / 1024
```



```
# 步骤 11: 返回报告表格中需要填写的核心指标。
return {
    "vector_dim": d,
    "dtype": str(base_vectors.dtype),
    "base_count": nb,
    "base_vector_size_mb": base_size_mb,
    "query_count": nq,
    "index_method": "IndexIVFFlat",
    "index_params": f"nlist={nlist}, nprobe={nprobe}",
    "build_time_ms": build_time_ms,
    "index_size_mb": index_size_mb,
    "qps": qps,
    "recall_at_10": recall10,
}
```

2.4 步骤 4: 整理实验结果表格

实验结果需要整理为表格。必须包含以下字段：

- 数据集名称；
- 使用的 embedding 模型；
- 向量维度；
- 向量数据类型；
- base 向量数量；
- base 向量大小；
- query 向量数量；
- 索引方法；
- 索引主要参数；
- 索引构建时间；
- 索引占用空间；
- QPS；
- Recall@10。

表格模板如下。

指标	第一组	第二组
数据集名称	Oxford-IIIT Pet	Oxford-IIIT Pet
Embedding 模型	CLIP ViT-B/32	CLIP ViT-B/32
向量维度	768	768
向量数据类型	float32	float32
Base 向量数量	7380	7380
Base 向量大小 MB	21.62	21.62
Query 向量数量	10	10
索引方法	IndexIVFFlat	IndexHNSWFlat

指标	第一组	第二组
索引主要参数	nlist=100, nprobe=10	M=32, efConstruction=80, efSearch=64
索引构建时间 ms	125.30	183.23
索引占用空间 MB	22.10	44.56
QPS	980.50	1829.39
Recall@10	0.90	0.95

说明:

- 上表仅为填写格式示例，具体数值以实际实验结果为准；
- 索引主要参数需要填写对应方法中影响检索效果和性能的关键参数,例如 IVF 的 nlist、nprobe, HNSW 的 M、efConstruction、efSearch, LSH 的 nbits;
- 向量大小、索引大小可以用文件大小或内存大小计算，但需要在报告中说明计算方式；
- QPS 计算方式为 $\text{query_num} / \text{search_time_seconds}$;
- Recall@10 需要与 ground truth top10 比较。

2.5 步骤 5: 检索结果可视化

任选一条 query，将该 query 的检索结果整理成可视化示例。报告中至少提供一页类似如下形式的结果图。

推荐布局如下:

- 第 1 行: Query 图片
- 第 2 行: Ground Truth Top-10 图片
- 第 3 行: 方法 A Top-10 图片, 例如 IVFFlat
- 第 4 行: 方法 B Top-10 图片, 例如 HNSW
- 第 5 行: 方法 C Top-10 图片, 例如 LSH, 可选

每一行的形式建议如下:

Query

[query image]

Ground Truth Top-10

[gt1] [gt2] [gt3] [gt4] [gt5] [gt6] [gt7] [gt8] [gt9] [gt10]

IVF Top-10 | Recall@10 = 0.90

[ivf1] [ivf2] [ivf3] [ivf4] [ivf5] [ivf6] [ivf7] [ivf8] [ivf9] [ivf10]

HNSW Top-10 | Recall@10 = 0.90

[hns1] [hns2] [hns3] [hns4] [hns5] [hns6] [hns7] [hns8] [hns9] [hns10]

报告中需要简要说明该 query 的可视化结果，例如：

- 哪种方法与 ground truth 最接近；
- 是否存在明显不符合人眼直觉的结果；
- 若存在不符合直觉的结果，可能是 embedding 模型造成的，还是 ANNS 近似搜索造成的。

3 Bonus

以下内容可选做任务，同学可以根据自己的时间和能力选择完成。

Bonus 1 (★)：使用不同 embedding 模型完成上述流程

在主任务的基础上，使用多种 embedding 模型完成相同的图像向量检索流程。例如：

- ResNet50；
- CLIP ViT-B/32；
- DINOv2；
- SigLIP。

回答核心问题是：不同 embedding 模型会不会改变“相似图片”的定义？同学需要结合不同 embedding 模型下的可视化查询结果，对该问题进行分析回答。例如，可以比较同一条 query 在不同 embedding 模型下返回的 top10 图片是否相同，以及哪种模型的结果更符合自己的直观感受。

Bonus 2 (★)：将图搜图流程改成文本搜图

通过选用图文多模态 embedding 模型，可以将图搜图流程扩展为文本搜图。

图搜图流程中，query 是一张图片；文本搜图流程中，query 是一段自然语言描述。只要使用 CLIP、SigLIP 等支持图文统一向量空间的模型，就可以将文本和图片映射到同一个向量空间中进行检索。

示例 query：

- 一只草地上的布偶猫；
- 一只黑色小狗；
- 一只躺在沙发上的猫；
- a dog running on grass。

同学需要展示若干条文本 query 的 top10 检索结果，并简要分析文本搜图结果是否符合语义描述。

Bonus 3 (★)：比较 embedding 归一化对检索结果的影响

在向量检索中，embedding 是否进行归一化可能会影响相似度计算方式和最终检索结果。本任务要求同学在相同 embedding 模型下，对比以下两种设置：

- 对 embedding 做归一化，例如使用 L2 normalization；
- 不对 embedding 做归一化，直接使用原始向量进行检索。

同学需要在相同的 base 图片库和 query 图片上完成检索，并比较两种设置下的 top10 返回结果是否发生变化。可以从以下角度进行分析：

- 同一条 query 在归一化和不归一化时返回的 top10 图片是否完全相同；

- 排名顺序是否发生变化；
- 哪一种结果更符合自己的直观感受；
- (optional) 当使用内积、余弦相似度或 L2 距离时，归一化对检索结果的影响是否不同。

回答核心问题是：`embedding` 归一化是否会改变图像检索中“相似图片”的排序结果？如果会，可能原因是什么？

Bonus 4 (★★): 将 FAISS 检索库换成开源向量数据库

主任务使用 FAISS 等 ANNS 库完成即可。选做任务可以将向量写入完整的开源向量数据库，并完成检索。

注意：这里的向量数据库不是普通 ANNS 库，而是独立的数据管理系统，通常支持 `collection`、`payload`、`filter`、持久化和服务接口等能力。

可选向量数据库如下：

向量数据库	部署难度	推荐用途	说明
Qdrant	低	图像检索、metadata filter、语义搜索	支持本地 Docker 部署，也支持 Python local mode。适合将图片路径、类别等信息作为 payload 与向量一起存储。参考：Qdrant Quickstart
Milvus Lite	低到中	本地实验、向量数据库入门	Milvus Lite 是 Milvus 的轻量版本，可在本地 Python 应用中运行，适合小规模实验、notebook 和 demo。参考：Milvus Lite 文档
Chroma	低	本地知识库、RAG 原型、轻量向量存储	部署简单，适合快速构建本地检索原型。适合扩展到文档检索和 RAG 场景。参考：Chroma 官方入门文档

要求：

1. 将 base 向量写入向量数据库；
2. payload 中保存图片路径、类别等信息；
3. 使用 query 向量检索 top10；
4. 回答核心问题：比较 FAISS 检索库和完整向量数据库在工程使用上的区别。

4 报告提交要求

本作业最终只需要提交实验报告 PDF，不需要提交代码文件。

报告需要至少包含以下内容：

1. 开头说明本次实验每个流程的选择，包括：
 - 数据集选择；
 - `embedding` 模型选择；
 - 选择的 ANNS 索引方法。

2. 列举选择的 Bonus 项目。若未选择 Bonus, 可以写“无”; 若选择, 需要在报告中详细展示 bonus 的完成情况。
3. 实验结果统计表格, 示例见 2.4 节步骤 4。表格需要包含:
 - 数据集名称;
 - embedding 模型;
 - 向量维度;
 - 向量数据类型;
 - base 向量数量;
 - base 向量大小;
 - query 向量数量;
 - 索引方法;
 - 索引主要参数;
 - 索引构建时间;
 - 索引占用空间;
 - QPS;
 - Recall@10。
4. 一条 query 的检索结果可视化。至少需要包括:
 - Query 图片;
 - Ground Truth Top-10 图片;
 - 至少两种 ANNS 方法的 Top-10 检索结果。
5. 开放性结果分析, 自选 3 个方面。可以参考以下角度展开:
 - 向量检索的优缺点
 - 对整个图像向量检索流程进行分析;
 - 对不同 embedding 模型的结果进行比较;
 - 对不同 ANNS 方法的 Recall@10、QPS、索引大小进行比较;
 - 分析某些 query 的结果为什么不符合人眼直觉;
 - 分析当前流程的不足以及可能的改进方向。
6. 遇到的问题及解决方案, 可选。如果实验过程中遇到环境配置、模型下载、向量维度不一致、索引训练失败等问题, 可以记录问题和解决过程。

A 附录: fvecs 与 ivecs 存储格式

fvecs 格式

fvecs 是常见的浮点向量二进制存储格式。每条向量前先存储一个 `int32` 类型的维度值 `d`, 后面紧跟 `d` 个 `float32` 数值。

```
[int32: d][float32: x1][float32: x2] ... [float32: xd]
[int32: d][float32: x1][float32: x2] ... [float32: xd]
...
```


ivecs 格式

ivecs 常用于保存 ground truth 或检索结果编号。每条记录前先存储一个 int32 类型的长度值 k , 后面紧跟 k 个 int32 编号。

```
[int32: k][int32: id1][int32: id2] ... [int32: idk]
[int32: k][int32: id1][int32: id2] ... [int32: idk]
...
```

B 附录: fvecs 与 ivecs 读写函数

```
from pathlib import Path
import numpy as np

def read_fvecs(path):
    """读取 fvecs 文件, 返回 shape 为 [N, d] 的 float32 向量矩阵。"""
    path = Path(path)

    # 步骤 1: 按 float32 读取整个二进制文件。
    raw = np.fromfile(path, dtype="float32")
    if raw.size == 0:
        raise RuntimeError(f"empty fvecs file: {path}")

    # 步骤 2: 每条 fvecs 记录的第一个 4 字节是 int32 维度 d。
    d = raw.view("int32")[0]

    # 步骤 3: 检查文件长度是否符合 [d + d 个 float32] 的重复结构。
    if raw.size % (d + 1) != 0:
        raise RuntimeError("invalid fvecs file format")

    # 步骤 4: reshape 后去掉每行第一个维度字段, 只保留真正的向量值。
    raw = raw.reshape(-1, d + 1)
    return raw[:, 1:].astype("float32")

def write_fvecs(path, vectors):
    """将 shape 为 [N, d] 的 float32 向量矩阵写入 fvecs 文件。"""
    path = Path(path)

    # 步骤 1: 确保写入的数据类型为 float32。
    vectors = np.asarray(vectors, dtype="float32")
    n, d = vectors.shape

    # 步骤 2: 逐条写入, 每条记录先写 int32 维度 d, 再写 d 个 float32。
    with open(path, "wb") as f:
```



```
        for i in range(n):
            np.array([d], dtype="int32").tofile(f)
            vectors[i].astype("float32").tofile(f)

def read_ivecs(path):
    """读取 ivecs 文件, 返回 shape 为 [N, k] 的 int32 编号矩阵。"""
    path = Path(path)

    # 步骤 1: ivecs 中所有内容都是 int32。
    raw = np.fromfile(path, dtype="int32")
    if raw.size == 0:
        raise RuntimeError(f"empty ivecs file: {path}")

    # 步骤 2: 每条记录的第一个 int32 是该行长度 k。
    k = raw[0]

    # 步骤 3: 检查文件长度是否符合 [k + k 个 id] 的重复结构。
    if raw.size % (k + 1) != 0:
        raise RuntimeError("invalid ivecs file format")

    # 步骤 4: reshape 后去掉每行第一个长度字段, 只保留编号。
    raw = raw.reshape(-1, k + 1)
    return raw[:, 1:].astype("int32")

def write_ivecs(path, indices):
    """将 shape 为 [N, k] 的 int32 编号矩阵写入 ivecs 文件。"""
    path = Path(path)

    # 步骤 1: 确保 ground truth 编号为 int32。
    indices = np.asarray(indices, dtype="int32")
    n, k = indices.shape

    # 步骤 2: 逐条写入, 每条记录先写 int32 长度 k, 再写 k 个 int32 编号。
    with open(path, "wb") as f:
        for i in range(n):
            np.array([k], dtype="int32").tofile(f)
            indices[i].astype("int32").tofile(f)
```